

2-Sprint Development Plan

Campus Events App — Onboarding, System & Organizer Epics

21

3

4

12

user stories

epics

team members

days to Apr 7

Deadline: April 7 · Sprint 1 runs days 1–6 · Sprint 2 runs days 7–12

1. Epic order & rationale

The three epics in this plan are sequenced by dependency: each one builds on the previous. Onboarding must come first because every other feature — browsing, ticketing, organizer tools — requires authenticated users. System stories (domain whitelisting and duplicate prevention) run in parallel with Onboarding because they do not depend on auth being complete; however they must be finished before organizers can register and before ticket logic is safe. The Organizer epic is placed third because events must exist before Discovery or Ticketing have anything to work with.

1	Onboarding	6	Foundation — nothing works without login
2	System	4	Runs in parallel with Onboarding; guards data integrity
3	Organizer	11	Events must exist before Discovery or Ticketing
4	Discovery	8	Requires events created by Organizer epic
5	Ticketing	9	Requires live, discoverable events
6	Notification	4	Triggered by Ticketing and Organizer actions
7	Social	3	Purely additive — nothing depends on it

2. Sprint 1 — Days 1 to 6

The team splits into two pairs. M1 and M2 own all six Onboarding stories, working frontend and backend in parallel. M3 and M4 tackle the four System stories simultaneously. The goal is for both epics to be fully done by end of day 6 so Sprint 2 can start with a clean slate.

Sprint 1

Days 1–6 · Target: complete by end of day 6

Epics: Onboarding + System

10 user stories

Onboarding Epic — M1 (Frontend) + M2 (Backend)

M1 and M2 work as a pair. M2 must deliver working API endpoints before M1 can wire up the UI — so the critical coordination point is agreeing on API contracts by day 2. Story #18 is shared: M2 owns the backend validation logic, M1 owns the form UI. Story #21 (organizer registration backend) is a key handoff point — once M2 finishes it, M3 can start implementing the domain whitelist check in #22 and #48.

M1 Aliza Khan Member 1 · Frontend — screens and flows			
#1	Choose between Campus User and Organizer role	Low risk	Frontend
#18	Registration form with university email (UI) — Coordinate API contract with M2 by day 2	Med risk	Frontend
#19	Password reset screen with OTP entry (UI) — Depends on M2 finishing OTP endpoint first	Med risk	Frontend
Note: Do not start wiring up forms until M2 has confirmed the API shape. Use day 1 to build static screens.			

M2 Mir Huzaifa Gujjar Member 2 · Backend — auth and registration logic			
#2	Login with university email — token handling	Med risk	Backend
#18	Registration backend — email validation + duplicates — Agree on response shape with M1 early	Med risk	Backend
#21	Organizer society/club registration backend — Feeds into M3's #22 and #48 — finish by day 4	Med risk	Backend
#19	OTP generation + email delivery for password reset	Med risk	Backend
Note: Prioritise #2 (login) and #18 (registration) first — they unblock M1's UI work. Finish #21 by day 4 to unblock M3.			

System Epic — M3 (Backend) + M4 (Backend)

M3 and M4 can start on day 1 independently of the Onboarding pair. M3's two stories (#48 and #22) are tightly coupled — they should be treated as a single unit of work. The biggest external risk here is obtaining the LUMS society domain list from LUMS IT; M3 must send that request on day 1, not mid-sprint. M4 has a lighter load — once #15 and #16 are done, M4 should assist M3 on the whitelist logic.

M3 Emaan Anwer Qureshi Member 3 · Backend — domain validation			
#48	Validate organizer email against LUMS domain whitelist — Source domain list from LUMS IT on day 1	High risk	Backend
#22	Block registration unless valid LUMS society domain — Treat as one piece of work with #48	High risk	Backend
Note: Treat #48 and #22 as one piece of work. External risk: domain list from LUMS IT. Email them day 1.			

M4 Muhammad Mahad Bhatti Member 4 · Backend — data integrity			
#15	Prevent duplicate form submissions without payment	Med risk	Backend
#50	Auto-close ticket sales when capacity is reached	Med risk	Backend
#16	Separate schema tracking for referral/discount codes — Quick win — assist M3 once done	Low risk	Backend
<i>Note: #16 is a quick schema task. Once done, assist M3 on whitelist logic to de-risk the sprint.</i>			

3.

Sprint 2 — Days 7 to 12 (due April 7)

All four members converge on the Organizer epic — the largest epic with 11 stories. The team works together for the first time because the volume requires it. Stories are distributed by skill area: M1 takes the dashboard UI, M2 handles core event creation and attendance data, M3 owns ticket and capacity logic, and M4 tackles the two hardest stories in the entire backlog.

Sprint 2	11 user stories
Days 7–12 · Deadline: April 7	
Epics: Organizer	

Story dependency chain within the Organizer epic

M2's story #10 (create event) is the anchor for the entire sprint — it defines the event schema that every other story reads from or writes to. M1 cannot fully wire up the dashboard (#9) until the event model exists. M3 cannot implement ticket tiers (#11) or capacity (#12) until the event record has the right fields. M4's QR scanning (#38) depends on ticket records being created, which depends on #11 and #12 being done. The recommended sequence is therefore: M2 starts #10 on day 7 and targets completion by day 9 — this is the unlock for M1, M3, and M4 to integrate their work against a real data model.

Day 7–9 #10 Create event (M2) — unlocks everything else in the sprint

Day 7–9 #38 QR scanning spike (M4) — start immediately, do not wait

Day 8–10 #9 Dashboard UI (M1), #11 Ticket price (M3), #12 Capacity (M3) — integrate with #10 once schema is confirmed

Day 10–12 #13 Sales timing (M2), #14 RSVP list (M2), #36 Edit event (M1), #35 Cover image (M1), #39 CSV export (M3), #37 Cancel event (M4) — all wrap up the sprint

Organizer Epic — story assignments

M1 Aliza Khan Member 1 · Frontend — dashboard and event UI			
#9	Organizer dashboard login screen UI — Build first — all dashboard views depend on this	Low risk	Frontend
#36	Edit event details before going live (UI)	Low risk	Frontend
#35	Upload cover image for event (UI + storage)	Med risk	Frontend
Note: Start with #9 (dashboard shell) on day 7 so other screens have a home. #35 requires a file storage decision — confirm with M2 which service to use.			

M2 Mir Huzaifa Gujjar Member 2 · Backend — core event model and data			
#10	Create event — title, description, date, venue — Most foundational story of the epic — start day 7	Low risk	Backend
#13	Set ticket sales start and end time — Depends on #10 schema being in place	Low risk	Backend
#14	View RSVP count and attendee list	Low risk	Backend
Note: #10 is the most critical story of the sprint. Target completion by day 9 to unblock M1, M3, and M4.			

M3 Emaan Anwer Qureshi Member 3 · Backend — ticketing and capacity logic			
#11	Set ticket price and quantity per ticket type — Do back-to-back with #12	Low risk	Backend
#12	Set event capacity to prevent overselling — Sync with M2 on event schema from #10	Med risk	Backend
#39	Export attendee list as CSV — Quick win — fills remaining days	Low risk	Backend
Note: #11 and #12 are tightly coupled — implement together. #39 is a quick CSV generation task to fill remaining time.			

M4 Muhammad Mahad Bhatti Member 4 · Senior — integration and high-risk stories			
#38	Scan attendee QR codes at gate for entry — Hardest story — camera, real-time validation, offline fallback. Start immediately day 7	High risk	Backend
#37	Cancel event and notify all attendees — Triggers bulk notification — handle partial failures	Med risk	Backend
Note: #38 is the riskiest story: camera access, real-time QR validation, offline fallback for poor connectivity. Spike on day 7, flag blockers immediately.			

After April 7 — remaining epics

With 21 stories done by April 7, the remaining 24 stories span four epics. The recommended order continues the dependency chain established in this plan.

Discovery	8	Events now exist — users can browse, search, and filter
Ticketing	9	Discovery exposes events; Ticketing lets users pay and RSVP
Notification	4	Triggered by ticket purchases and event cancellations
Social	3	Additive layer — wishlists, sharing, attendance counts

The two highest-risk stories in the remaining backlog are **#6** (Paymo payment integration) and **#34** (duplicate ticket prevention under concurrent load). Both should be spiked at the start of the Ticketing sprint, not left until the end.